

NATIONAL ECONOMICS UNIVERSITY
Faculty of Data Science and Artificial Intelligence
MIDTERM EXAMINATION
DATA ANALYSIS WITH PYTHON (DSAI1005)
EXAM VERSION: 301-EN

Full name: **Student ID:**
Class: **Room:**
Signature: **Exam date:**

Duration: 60 minutes (1 hour). **Total:** 10.0 points.

The exam has two parts: **(I) Multiple Choice** — 6.0 points (60%); **(II) Integrated Data Problems** — 4.0 points (40%).

INSTRUCTIONS

1. Write directly on this exam paper and keep each answer inside the designated answer area.
2. For output-prediction questions, reason from the code; you do not need to run Python.
3. For coding questions, write only the necessary statements; do not copy the given data again.
4. Equivalent correct solutions are accepted when the logic and result are correct.
5. Show clear reasoning where requested. Partial credit may be awarded for a correct method even if a final arithmetic value is slightly inaccurate.

Part	Part I (Multiple Choice)	Part II (Integrated Problems)	Total
Maximum	6.0	4.0	10.0
Score			

Invigilators do not provide additional explanation of technical content.

PART I — MULTIPLE CHOICE (12 questions × 0.5 = 6.0 points)

Select one best answer for each question and mark A/B/C/D directly on this paper.

1. Consider the following NumPy array initialization:

```
import numpy as np
a = np.linspace(0, 10, 5)
b = np.arange(1, 6)
res = a + b * 2
```

What is the resulting array res?

- A. [2., 6.5, 11., 15.5, 20.]
- B. [2., 4.5, 9., 13.5, 18.]
- C. [2, 7, 11, 16, 20]
- D. [0., 4.5, 11., 15.5, 22.]

Answer: ☐ A ☐ B ☐ C ☐ D

2. Basic slicing creates a memory view rather than an independent copy:

```
m = np.array([[10, 20, 30],
              [40, 50, 60],
              [70, 80, 90]])
v = m[:, -1, 1]
v[0] = 999
print(m[2, 1], v[2])
```

What values are printed to the console?

- A. 80 20
- B. 999 20
- C. 80 999
- D. 999 80

Answer: ☐ A ☐ B ☐ C ☐ D

3. Array *A* has shape (4, 1, 6) and array *B* has shape (3, 1). Under NumPy broadcasting rules, what is the shape of *C* = *A* * *B* + np.zeros(6)?

- A. (4, 3, 6)
- B. (4, 1, 6)
- C. (3, 4, 6)
- D. Raises ValueError

Answer: ☐ A ☐ B ☐ C ☐ D

4. Consider conditional filtering and axis reduction:

```
x = np.array([[12, 25, 8], [30, 15, 22]])
filtered = np.where((x % 2 == 0) & (x > 10),
                   x * 2, 0)
print(np.sum(filtered, axis=0))
```

What output array is displayed?

- A. [84, 0, 44]
- B. [24, 60]
- C. [84, 50, 44]
- D. [24, 0, 44]

Answer: ☐ A ☐ B ☐ C ☐ D

5. When performing arithmetic between two Series *S*₁ and *S*₂ with non-identical string index labels:

```
import pandas as pd
s1 = pd.Series([10, 20, 30],
               index=['Hanoi', 'HCMC', 'Da Nang'])
s2 = pd.Series([5, 15, 25],
               index=['HCMC', 'Can Tho', 'Hanoi'])
res = s1.add(s2, fill_value=0)
```

What are res['Da Nang'] and res['Can Tho']?

- A. Both evaluate to NaN.
- B. 30.0 and 15.0 respectively.
- C. An IndexMismatchError is raised.
- D. 0.0 and 0.0 respectively.

Answer: ☐ A ☐ B ☐ C ☐ D

6. Consider the sales DataFrame df:

```
df = pd.DataFrame({
    'Revenue': [150, 280, 310, 450],
    'Cost': [100, 190, 200, 300]
}, index=['Q1', 'Q2', 'Q3', 'Q4'])
loc_slice = df.loc['Q2': 'Q4', 'Revenue']
iloc_slice = df.iloc[1:3, 0]
```

Number of elements in loc_slice and iloc_slice:

- A. loc_slice: 3; iloc_slice: 2
- B. loc_slice: 2; iloc_slice: 2
- C. loc_slice: 3; iloc_slice: 3
- D. loc_slice: 2; iloc_slice: 3

Answer: ☐ A ☐ B ☐ C ☐ D

PART I — MULTIPLE CHOICE (continued)

Select one best answer for each question and mark A/B/C/D directly on this paper.

7. Given the multi-level aggregation pipeline on retail orders:

```
orders = pd.DataFrame({
    'Region': ['North', 'South', 'North',
              'South', 'North'],
    'Category': ['Food', 'Food', 'Tech',
                'Tech', 'Food'],
    'Sales': [100, 150, 300, 250, 200]
})
summary = orders.groupby(['Region', 'Category']
                        )['Sales'].agg(['sum', 'count'])
print(summary.loc(['North', 'Food'], 'sum'))
```

What value is printed to the console?

- A. 100
- B. 300
- C. 2
- D. 450

Answer: ☐ A ☐ B ☐ C ☐ D

8. You execute a Left Merge between table A ($N = 500$ rows) and table B ($M = 80$ rows) on key 'store_id':

```
merged = pd.merge(A, B, on='store_id', how='left')
```

Assuming 'store_id' is unique in table B, but 30 rows in A have no match in B. Which statement is true?

- A. merged contains exactly 470 rows.
- B. merged contains exactly 500 rows, with NaN for table B columns on the 30 unmatched rows.
- C. merged contains exactly 80 rows.
- D. A KeyError is thrown.

Answer: ☐ A ☐ B ☐ C ☐ D

9. In Matplotlib's Object-Oriented interface:

```
import matplotlib.pyplot as plt
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
```

Which statement correctly configures the subplot in the **first row, second column**?

- A. axes[0, 1].set_title("Branch Q2")
- B. axes[1, 2].set_title("Branch Q2")
- C. axes[1, 0].title = "Branch Q2"
- D. plt.subplot(1, 2).set_title("Branch Q2")

Answer: ☐ A ☐ B ☐ C ☐ D

10. When analyzing continuous financial transaction values with extreme high-value outliers and severe positive skewness, which visualization tool is most appropriate for observing the median, *IQR*, and outlier whiskers simultaneously?

- A. sns.barplot()
- B. sns.boxplot()
- C. sns.countplot()
- D. sns.lineplot()

Answer: ☐ A ☐ B ☐ C ☐ D

11. Consider the Seaborn scatter plot statement:

```
import seaborn as sns
sns.scatterplot(data=df, x='Marketing_Spend',
               y='Revenue', hue='Channel',
               style='Tier')
```

What roles do hue and style play?

- A. hue maps channel categories to distinct colors; style maps tier categories to distinct marker shapes.
- B. hue adjusts opacity; style sets font size.
- C. hue creates subplots; style sets line width.
- D. Both parameters are identical synonyms.

Answer: ☐ A ☐ B ☐ C ☐ D

12. Given the correlation matrix visualization:

```
corr_matrix = df[['Age', 'Income', 'Score']].corr()
sns.heatmap(corr_matrix, annot=True,
            cmap='coolwarm', vmin=-1, vmax=1)
plt.tight_layout()
```

Primary role of `annot=True` and `plt.tight_layout()`:

- A. `annot=True` adds tooltips; `tight_layout()` downsamples resolution.
- B. `annot=True` writes correlation values in cells; `tight_layout()` adjusts subplot margins.
- C. `annot=True` filters zeros; `tight_layout()` converts to grayscale.
- D. `annot=True` computes p-values; `tight_layout()` locks 1:1 aspect ratio.

Answer: ☐ A ☐ B ☐ C ☐ D

PART II — INTEGRATED DATA PROBLEMS (4.0 points)

Question 13. [2.0 points] Quantitative E-Commerce Performance Pipeline [NumPy & Pandas]

Scenario: An enterprise processes transactional batches. Given raw transactional array `tx_data` (5×4), where columns represent [0: unit_price, 1: quantity, 2: discount_rate (0 to 1), 3: shipping_cost]:

```
import numpy as np
import pandas as pd

tx_data = np.array([
    [50000.0, 3.0, 0.10, 15000.0], [120000.0, 1.0, 0.00, 20000.0],
    [45000.0, 4.0, 0.05, 10000.0], [300000.0, 2.0, 0.20, 25000.0],
    [80000.0, 5.0, 0.15, 15000.0]
])
```

a) [0.40] Write a vectorized NumPy expression to compute 1D array `net_revenue` across all 5 orders according to the formula: $\text{net_revenue} = (\text{unit_price} \times \text{quantity} \times (1 - \text{discount_rate})) - \text{shipping_cost}$.

Write your answer in this box.

b) [0.40] Using `np.where`, write an expression to create a 1D string array `order_tier` categorizing an order as "Wholesale" if $\text{net_revenue} \geq 250\,000$ **or** $\text{quantity} \geq 4$, otherwise "Retail".

Write your answer in this box.

c) [0.40] Assume DataFrame `orders_df` (columns: `order_id`, `cust_id`, `net_rev`, `tier`) and customer table `cust_df` (columns: `cust_id`, `city`, `status`) are defined. Write a Pandas statement to merge `orders_df` with `cust_df` on '`cust_id`' such that **all orders are retained** (a Left Merge).

Write your answer in this box.

d) [0.40] On the merged DataFrame `merged`, write a statement chaining `.groupby()` and `.agg()` to group records by ['`city`', '`tier`'] and compute both the **sum** and **count** of '`net_rev`'.

Write your answer in this box.

e) [0.40] Write a statement to sort the resulting aggregated DataFrame `summary` by total revenue (the sum of `net_rev`) in **descending** order.

Write your answer in this box.

Question 14. [2.0 points] Sales Dashboard & Anomaly Visualization [Pandas & Viz]

Scenario: You are constructing an executive analytics dashboard comparing sales distributions across payment channels and monitoring risk anomalies. You are provided with a pre-loaded DataFrame `df_sales`:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Schema of df_sales:
# 'order_date': string formatted dates (e.g. '2026-09-01')
# 'channel': string ('Online', 'Store', 'Partner')
# 'payment': string ('Credit Card', 'Cash', 'Bank Transfer')
# 'amount': float (transaction order value in VND)
# 'is_fraud': int (1 for suspicious/fraudulent, 0 for legitimate)
```

a) [0.40] Write a Matplotlib Object-Oriented statement to initialize a figure and a 1×2 subplot grid (1 row, 2 columns) named `fig`, axes with overall figure dimensions of 14×5 inches.

Write your answer in this box.

b) [0.60] Write a complete Seaborn call to render a grouped Boxplot on `axes[0]` displaying transaction amounts (`y='amount'`) across payment methods (`x='payment'`), stratified by fraud status (`hue='is_fraud'`), using color palette 'Set2', plotted specifically onto target axis `axes[0]`.

Write your answer in this box.

After plotting, the left subplot is labeled as follows:

```
axes[0].set_title("Transaction Amount Distribution by Payment Method", fontsize=12)
axes[0].set_xlabel("Payment Method", fontsize=10)
axes[0].set_ylabel("Order Value (VND)", fontsize=10)
```

QUESTION 14 — CONTINUED

On the second subplot (`axes[1]`), order dates are parsed and indexed to analyze weekly revenue trajectory:

```
df_sales['order_date'] = pd.to_datetime(df_sales['order_date'])
df_sales = df_sales.set_index('order_date')
weekly_sales = df_sales['amount'].resample('W').sum()
```

c) [0.40] Write a Pandas statement using `weekly_sales` to compute a **4-week rolling moving average** named `rolling_trend` to smooth erratic short-term revenue variance.

Write your answer in this box.

Both actual weekly sales and the smoothed rolling trend are plotted on `axes[1]`:

```
axes[1].plot(weekly_sales.index, weekly_sales.values, label='Weekly Actual', color='steelblue', alpha=0.6)
axes[1].plot(weekly_sales.index, rolling_trend.values, label='4-Week Trend', color='crimson', linewidth=2)
axes[1].set_title("Weekly Revenue Trajectory & 4-Week Moving Average", fontsize=12)
axes[1].set_xlabel("Week Ending Date", fontsize=10); axes[1].set_ylabel("Total Revenue (VND)", fontsize=10)
axes[1].legend(); axes[1].grid(True, linestyle='--', alpha=0.5)
```

d) [0.20] Write the standard Matplotlib command to automatically adjust canvas padding and prevent subplot titles and tick labels from overlapping.

Write your answer in this box.

e) [0.20] In the boxplot on `axes[0]`, how can a risk analyst visually differentiate between typical legitimate transactions and fraudulent outliers in terms of the interquartile range (*IQR*) and whisker boundaries?

Write your answer in this box.

f) [0.20] Why is plotting a 4-week rolling trend line on `axes[1]` analytically superior to observing only erratic week-to-week revenue spikes when forecasting quarterly revenue?

Write your answer in this box.

— END OF EXAMINATION PAPER —